

Guia de entrevista tecnica: Deploy de Premier Hub

Banco de preguntas posibles con respuesta oral sugerida y evidencia esperada, aterrizado al backend y frontend del proyecto Premier Hub.

Ambientes: `preprod` y `prod`

Plataforma: Docker + K3s + GitHub Actions self-hosted

Backend: Express/TypeScript en puerto `4000`

Frontend: React/Vite servido con Nginx

Resumen ejecutivo para abrir la entrevista

Respuesta de 30 segundos: Nuestro despliegue es multientorno porque tenemos una rama para `preprod` y otra para `main/prod`. Cada push dispara GitHub Actions en un runner self-hosted. El pipeline construye una imagen Docker, la etiqueta con el SHA del commit, la importa al runtime de K3s y actualiza el Deployment de Kubernetes con `kubectl set image`. Despues valida el rollout con `kubectl rollout status`. El backend expone `/api/health`, usa variables de entorno para secretos y configuracion, y el frontend se compila con URLs distintas por ambiente.

Tema	Como aplica en Premier Hub	Evidencia esperada
Multientorno	<code>preprod</code> despliega a namespace <code>preprod</code> ; <code>main</code> despliega a <code>prod</code> .	<code>.github/workflows/deploy-preprod.yml</code> y <code>.github/workflows/deploy.yml</code> .
CI/CD	Pipeline automatico en push, build de imagen, import a K3s, rollout.	GitHub Actions con <code>runs-on: self-hosted</code> .
Contenedores	Backend en Node 20 Alpine; frontend multi-stage con Node + Nginx.	<code>api/Dockerfile</code> y <code>pagina/Dockerfile</code> .
Health check	Endpoint <code>GET /api/health</code> responde <code>{ status: "ok" }</code> .	<code>api/src/index.ts</code> .
Rollback	Kubernetes puede volver al ReplicaSet anterior o redeployar un SHA previo.	<code>kubectl rollout undo deployment/api -n prod</code> .
Secretos	Backend lee llaves con <code>process.env</code> ; deben vivir en Secrets.	<code>SUPABASE_SERVICE_KEY</code> , <code>COOKIE_SECRET</code> , <code>NEWS_API_KEY</code> .

Mapa real del deploy

Backend:

push a preprod -> GitHub Actions -> docker build premier-api:<sha> -> import a K3s -> deployment/api -n preprod

push a main -> GitHub Actions -> docker build premier-api:<sha> -> import a K3s -> deployment/api -n prod

Frontend:

push a preprod -> build con URLs preprod -> imagen premier-pagina:<sha> -> deployment/pagina -n preprod

push a main -> build con URLs prod -> imagen premier-pagina:<sha> -> deployment/pagina -n prod

Preguntas y respuestas

1. Explica el proceso completo de deploy del backend.

Respuesta:

El backend se despliega automáticamente con GitHub Actions. Cuando hacemos push a `preprod` o `main`, el runner self-hosted clona el repo, construye la imagen Docker desde `./api`, la etiqueta con el SHA del commit, la importa a K3s y actualiza el deployment `api` en el namespace correspondiente. Finalmente espera el rollout para confirmar que Kubernetes pudo levantar la nueva versión.

Evidencia esperada:

- `.github/workflows/deploy-preprod.yml`: rama `preprod`, namespace `preprod`.
- `.github/workflows/deploy.yml`: rama `main`, namespace `prod`.
- Comandos `docker build`, `docker save` | `k3s ctr images import`, `kubectl set image`.

2. Que significa que el deploy sea multientorno?

Respuesta:

Significa que el mismo sistema puede ejecutarse en ambientes separados con configuración, URLs y recursos independientes. En el proyecto, `preprod` sirve para validar cambios antes de producción y `prod` es el ambiente final para usuarios. La separación se ve en ramas, namespaces de Kubernetes y dominios del frontend/API.

Evidencia esperada:

- Backend: `deployment/api -n preprod` contra `deployment/api -n prod`.
- Frontend preprod: `https://api-preprod.zamer-o.com`.
- Frontend prod: `https://api.zamer-o.com`.

3. Cual es la diferencia entre preproduccion y produccion?

Respuesta:

Preproduccion es un ambiente casi igual a produccion, pero usado para probar integraciones, configuracion y despliegues antes de afectar usuarios reales. Produccion es el ambiente estable. En nuestro caso, un push a `preprod` actualiza el namespace `preprod`, y un push a `main` actualiza el namespace `prod`.

Evidencia esperada: workflows separados para `preprod` y `main`.

4. Que rama dispara cada ambiente?

Respuesta:

La rama `preprod` dispara el deploy de preproduccion. La rama `main` dispara el deploy de produccion.

Evidencia esperada: bloque `on: push: branches` en ambos workflows.

5. Que es CI/CD y donde aparece en el proyecto?

Respuesta:

CI/CD es automatizar la integracion y entrega/despliegue del software. En el proyecto, cada push a las ramas de ambiente ejecuta un pipeline que construye la imagen y despliega a Kubernetes. La parte de CI es validar que el codigo compile dentro de la imagen; la parte de CD es actualizar el deployment en K3s.

Evidencia esperada: GitHub Actions en `.github/workflows`.

6. Por que usan un runner self-hosted?

Respuesta:

Porque el pipeline necesita acceso directo al cluster K3s local o privado. Un runner hospedado por GitHub normalmente no tendria acceso a ese entorno ni al runtime de imagenes de K3s. El runner self-hosted corre dentro de la infraestructura donde puede ejecutar `docker`, `sudo k3s ctr` y `kubect1`.

Evidencia esperada: `runs-on: self-hosted` en los workflows.

7. Por que etiquetar la imagen con `github.sha` ?

Respuesta:

Porque el SHA identifica de forma unica el commit desplegado. Eso da trazabilidad: si algo falla, podemos saber exactamente que version esta en Kubernetes. Tambien ayuda al rollback, porque podemos regresar a una imagen anterior identificada por SHA.

Evidencia esperada: `-t premier-api:${{ github.sha }}` y `-t premier-pagina:${{ github.sha }}`.

8. Para que sirve la etiqueta `latest` o `preprod` ?

Respuesta:

Sirve como etiqueta humana o conveniente para identificar la version mas reciente de un ambiente. Sin embargo, el deployment realmente usa la etiqueta por SHA, que es mas segura porque es inmutable a nivel de version.

Evidencia esperada: el build genera dos tags, pero `kubectl set image` usa `${{ github.sha }}`.

9. Que hace `docker save` | `sudo k3s ctr images import` ?

Respuesta:

Como no se esta usando un registry externo en el workflow, la imagen se guarda desde Docker y se importa directamente al runtime de K3s. Asi Kubernetes puede encontrar la imagen localmente cuando se actualiza el deployment.

Evidencia esperada: linea `docker save premier-api:${{ github.sha }}` | `sudo k3s ctr images import -`.

10. Que hace `kubectl set image` ?

Respuesta:

Actualiza la imagen de un contenedor dentro de un Deployment de Kubernetes. En backend cambia el contenedor `api` del deployment `api`. En frontend cambia el contenedor `pagina` del deployment `pagina`. Eso dispara un rolling update.

Evidencia esperada: `kubectl set image deployment/api api=premier-api:<sha>`.

11. Que hace `kubectl rollout status` ?

Respuesta:

Espera a que Kubernetes termine el despliegue. Si los pods nuevos levantan bien, el comando termina correctamente. Si hay problemas, por ejemplo la app no arranca o la imagen no existe, el rollout no se completa dentro del timeout.

Evidencia esperada: `kubectl rollout status deployment/api -n prod --timeout=120s` .

12. Como esta construido el contenedor del backend?

Respuesta:

Usa `node:20-alpine` , copia `package.json` y `package-lock.json` , instala dependencias, copia `tsconfig.json` y `src` , compila TypeScript con `npm run build` , expone el puerto `4000` y arranca con `node dist/index.js` .

Evidencia esperada: `api/Dockerfile` .

13. Por que el backend escucha en `0.0.0.0` ?

Respuesta:

Porque dentro de un contenedor no basta con escuchar solo en `localhost` . Al usar `0.0.0.0` , Express acepta conexiones desde la red del contenedor, lo que permite que Kubernetes o el Service enruten trafico hacia la API.

Evidencia esperada: `app.listen(PORT, "0.0.0.0", ...)` en `api/src/index.ts` .

14. Como se configura el puerto del backend?

Respuesta:

El backend lee `process.env.PORT` . Si no existe, usa `4000` . Esto permite que el mismo codigo corra localmente y en Kubernetes con configuracion distinta.

Evidencia esperada: `const PORT = Number(process.env.PORT) || 4000` .

15. Como esta construido el contenedor del frontend?

Respuesta:

Es un build multi-stage. Primero usa Node para instalar dependencias y compilar React/Vite. Luego copia el resultado `dist` a una imagen `nginx:alpine` , que sirve los archivos estaticos en el puerto `80` .

Evidencia esperada: `front/premier-hub-frontend/pagina/Dockerfile` .

16. Por que el frontend usa build args?

Respuesta:

Vite inyecta variables `VITE_*` al momento de compilar. Por eso el pipeline pasa `VITE_API_URL`, `VITE_APP_URL` y `VITE_SUPABASE_URL` como build args. Así el bundle de preprod apunta a servicios preprod y el de prod apunta a servicios prod.

Evidencia esperada: workflows del frontend con URLs `api-preprod.zamer-o.com` y `api.zamer-o.com`.

17. Que hace la configuracion de Nginx del frontend?

Respuesta:

Sirve el frontend estatico y soporta rutas de React Router con `try_files $uri $uri/ /index.html`. También configura cache: evita cache fuerte para la app principal y permite cache largo para `/assets`, que normalmente son archivos versionados por hash.

Evidencia esperada: `pagina/nginx.conf`.

18. Que variables de entorno usa el backend?

Respuesta:

Usa variables para base de datos, sesiones, CORS y APIs externas: `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `COOKIE_SECRET`, `CORS_ORIGIN`, `DEV_CORS_ORIGIN`, `APIFOOTBALL_KEY`, `NEWS_API_KEY`, `NODE_ENV`, `PORT`, entre otras.

Evidencia esperada: busqueda de `process.env` en `api/src`.

19. Que diferencia hay entre Secret y ConfigMap?

Respuesta:

Un Secret se usa para datos sensibles como llaves, tokens y claves de cookies. Un ConfigMap se usa para configuracion no sensible como puertos, URLs publicas o banderas de ambiente. En Premier Hub, `SUPABASE_SERVICE_KEY`, `COOKIE_SECRET`, `NEWS_API_KEY` y `APIFOOTBALL_KEY` deberian ser Secrets. `PORT` o `CORS_ORIGIN` podrian ser ConfigMap, aunque CORS tambien puede tratarse con cuidado por seguridad.

Evidencia esperada: variables leidas en `api/src/db.ts`, `api/src/index.ts` y rutas.

20. Donde se conectan a Supabase?

Respuesta:

La conexión está centralizada en `api/src/db.ts`. Se crea un cliente Supabase con `SUPABASE_URL` y `SUPABASE_SERVICE_KEY`, usando el schema `premier`. El service key es sensible porque puede tener permisos elevados, por eso debe ir como Secret.

Evidencia esperada: `createClient(process.env.SUPABASE_URL!, process.env.SUPABASE_SERVICE_KEY!, ...)`.

21. Como manejan CORS?

Respuesta:

Express usa el middleware `cors`. El origen permitido viene de `CORS_ORIGIN` o `DEV_CORS_ORIGIN`, con fallback a `http://localhost:5173`. También habilita `credentials: true` porque la app usa cookies de sesión.

Evidencia esperada: configuración de `cors` en `api/src/index.ts`.

22. Como se maneja la autenticación de sesión?

Respuesta:

El backend usa una cookie llamada `ph_session`. La cookie es `httpOnly`, firmada con `COOKIE_SECRET`, tiene `sameSite: "lax"` y en producción se marca como `secure`. Esto reduce el riesgo de que JavaScript del navegador lea la sesión.

Evidencia esperada: `setSessionCookie` en `api/src/rutas/api_auth.ts`.

23. Que endpoint sirve como health check?

Respuesta:

El endpoint `GET /api/health`. Responde JSON con `{ status: "ok" }`. Sirve para comprobar que el proceso Express está vivo. Como mejora, podría validar también dependencias críticas como Supabase.

Evidencia esperada: ruta en `api/src/index.ts`.

24. Health check y readiness check son lo mismo?

Respuesta:

No. Un liveness check responde si el proceso esta vivo. Un readiness check responde si la app esta lista para recibir trafico. Nuestro `/api/health` es basico: confirma que Express responde, pero no comprueba si Supabase o APIs externas estan disponibles. Para produccion seria ideal separar `/health/live` y `/health/ready`.

Evidencia esperada: endpoint actual solo devuelve `status: "ok"`.

25. Como harias rollback si el deploy falla?

Respuesta:

Primero revisaria el rollout con `kubectl rollout status`. Si la version nueva falla, usaria `kubectl rollout undo deployment/api -n prod` para regresar al ReplicaSet anterior. Tambien puedo redeployar una imagen anterior porque las imagenes estan etiquetadas con SHA.

Evidencia esperada: despliegue con SHA y uso de Kubernetes Deployment.

26. Que pasa si el build de Docker falla?

Respuesta:

El pipeline se detiene antes de tocar Kubernetes. Eso evita desplegar una version que ni siquiera compila. En backend, `npm run build` ejecuta TypeScript; en frontend, el build ejecuta `tsc && vite build`.

Evidencia esperada: scripts `build` en `package.json` y Dockerfiles.

27. Que pasa si la imagen se construye pero el pod no levanta?

Respuesta:

Kubernetes intentara crear pods nuevos, pero el rollout no terminara correctamente. El workflow espera hasta 120 segundos. Para diagnosticar, revisaria `kubectl describe pod`, `kubectl logs` y eventos del namespace. Causas comunes: variables faltantes, error al arrancar Node, imagen no disponible o puertos mal configurados.

Evidencia esperada: `rollout status --timeout=120s`.

28. Como verificarias que el deploy funciono?

Respuesta:

Revisaria que el workflow haya terminado exitosamente, que el rollout este completo, que los pods esten en estado Running y que `/api/health` responda. Tambien probaria una ruta funcional como login, noticias o partidos dependiendo del cambio.

Evidencia esperada: GitHub Actions, `kubectl get pods -n prod`, `curl https://api.zamer-o.com/api/health`.

29. Que observabilidad tiene el proyecto?

Respuesta:

Actualmente tiene logs por consola, errores con `console.error`, health check y validacion de rollout en CI/CD. Eso da una base operativa, pero no es observabilidad completa. Como mejora, agregaria logs centralizados, metricas de latencia/error rate, dashboard y alertas.

Evidencia esperada: `console.log`, `console.error`, `/api/health`, `rollout status`.

30. Que metricas monitorearias en produccion?

Respuesta:

Monitorearia disponibilidad del health check, latencia p95, tasa de errores 4xx/5xx, CPU y memoria de pods, reinicios, tiempo de respuesta de Supabase, errores de APIs externas y duracion de rollout. Para producto, tambien monitorearia endpoints criticos: auth, ranking, noticias, partidos y live.

Evidencia esperada: rutas montadas en `api/src/index.ts`.

31. Que logs revisarias ante una falla?

Respuesta:

Revisaria logs del workflow, eventos de Kubernetes, logs del pod del backend y errores de Supabase o APIs externas. Si falla auth, revisaria cookies y variables de sesion. Si fallan noticias o partidos, revisaria las llaves `NEWS_API_KEY` o `APIFOOTBALL_KEY`.

Evidencia esperada: `kubectl logs deployment/api -n prod`, logs de GitHub Actions.

32. Que medidas de seguridad ya existen?

Respuesta:

Las sesiones usan cookies `httpOnly` y firmadas. En producción se usa `secure`. CORS está configurado por ambiente. Los secretos se leen por variables de entorno. Además, el frontend no debería recibir llaves privadas; solo recibe URLs públicas de ambiente.

Evidencia esperada: `api_auth.ts`, `index.ts`, Dockerfile del frontend con `VITE_*`.

33. Cual es el riesgo de `SUPABASE_SERVICE_KEY` ?

Respuesta:

Es una llave sensible del backend. Si se expone, alguien podría tener permisos elevados sobre la base de datos. Por eso solo debe vivir en el servidor, inyectada como Secret, nunca en el frontend, nunca en el repositorio y nunca en logs.

Evidencia esperada: `api/src/db.ts` usa `SUPABASE_SERVICE_KEY`; frontend solo recibe `VITE_SUPABASE_URL`.

34. Por que no se deben guardar secretos en Git?

Respuesta:

Porque Git conserva historial. Aunque borres un secreto del archivo, puede quedar en commits anteriores. La práctica correcta es usar variables de entorno, GitHub Secrets, Kubernetes Secrets o un gestor de secretos.

Evidencia esperada: el código usa `process.env` en vez de llaves hardcodeadas.

35. Que es infraestructura como código y que tiene el proyecto?

Respuesta:

Infraestructura como código significa versionar la definición de infraestructura, por ejemplo Deployments, Services, Ingress, Secrets templates o Terraform. En este repo si están versionados los Dockerfiles y workflows de CI/CD, pero no vi manifests de Kubernetes ni Terraform. Como mejora, agregaría manifests o Helm/Kustomize para que la infraestructura también sea reproducible desde Git.

Evidencia esperada: existen `Dockerfile` y `.github/workflows`; no aparecen `*.yaml` de Kubernetes en el repo.

36. Como manejan imagenes Docker?

Respuesta:

El pipeline construye imagenes locales y las etiqueta por SHA. Luego las importa al runtime de K3s. Para backend la imagen es `premier-api` ; para frontend es `premier-pagina` . Actualmente no se ve un registry externo; como mejora se podria usar GHCR o Docker Hub privado para trazabilidad, retencion y despliegues mas estandar.

Evidencia esperada: `docker build -t premier-api:${{ github.sha }}` .

37. Que ventaja tiene usar Kubernetes/K3s?

Respuesta:

Kubernetes administra el estado deseado: si actualizo la imagen de un Deployment, crea pods nuevos y reemplaza gradualmente los anteriores. K3s es una distribucion ligera de Kubernetes, util para proyectos o clusters mas pequenos. Nos da namespaces, rollouts y rollback.

Evidencia esperada: uso de `kubectl set image` y namespaces `preprod/prod` .

38. Como se separan los recursos entre ambientes?

Respuesta:

La separacion visible esta en namespaces: `preprod` y `prod` . Eso permite tener Deployments, Services y configuracion separados aunque usen nombres similares. Tambien el frontend apunta a dominios diferentes por ambiente.

Evidencia esperada: `-n preprod` y `-n prod` en workflows.

39. Que pasa con la base de datos en un deploy?

Respuesta:

El deploy de la aplicacion no recrea la base de datos. La app se conecta a Supabase por variables de entorno. Si hay cambios de schema, deben aplicarse de forma controlada con scripts SQL o migraciones. En el repo hay archivos SQL para tablas como cache de noticias y features live.

Evidencia esperada: carpeta `api/sql` y cliente Supabase en `api/src/db.ts` .

40. Como respaldarian datos?

Respuesta:

Como la base esta en Supabase/Postgres, el respaldo debe hacerse a nivel de base de datos: backups automaticos del proveedor, snapshots programados, exportaciones SQL o dumps antes de migraciones riesgosas. Para archivos en buckets, tambien se debe considerar respaldo o politica de retencion.

Evidencia esperada: uso de Supabase y buckets como `profilePictures` en auth.

41. Como harias una migracion segura de base de datos?

Respuesta:

Primero aplicaria la migracion en preprod, validaria que el backend funcione, haria backup antes de produccion y disenaria cambios compatibles hacia atras cuando sea posible. Por ejemplo, agregar una columna antes de hacer que el codigo dependa obligatoriamente de ella. Despues desplegaria app y schema en orden controlado.

Evidencia esperada: archivos SQL en `api/sql` ; separacion preprod/prod.

42. Que dependencias externas tiene la app?

Respuesta:

Depende de Supabase para datos y almacenamiento, API-Football para partidos, NewsAPI para noticias, y sitios externos para scraping de articulos. Estas dependencias afectan disponibilidad, por eso hay que manejar errores, timeouts y cache.

Evidencia esperada: `api_partidos.ts` , `api_noticias.ts` , `db.ts` .

43. Como reducen el impacto de APIs externas?

Respuesta:

En noticias hay cache en memoria y cache persistido en Supabase con expiracion. Tambien hay fallback: si falla refrescar noticias pero existe una version cacheada, se puede responder cache stale. Eso mejora resiliencia y reduce llamadas a APIs externas.

Evidencia esperada: `NEWS_CACHE_TTL_MS` , `readNewsSnapshot` , `persistNewsSnapshot` , `fallback: true` .

44. Que riesgos ves en el deploy actual?**Respuesta:**

El flujo funciona, pero tiene mejoras posibles: no se ven tests automatizados antes del deploy, no se ven manifests de Kubernetes versionados, no se ve registry externo, el health check es basico, y la observabilidad podria ser mas completa. Tambien revisaria que Secrets y ConfigMaps esten bien configurados en el cluster.

Evidencia esperada: workflows solo tienen build/deploy; no hay manifests Kubernetes en el repo.

45. Que mejorarias si tuvieras mas tiempo?**Respuesta:**

Agregaria pruebas automatizadas al pipeline, manifests Kubernetes versionados con Helm o Kustomize, registry de imagenes, health checks separados para liveness/readiness, monitoreo con metricas, alertas, politicas de backup documentadas y escaneo de vulnerabilidades de imagenes.

Evidencia esperada: mejora propuesta basada en gaps reales del repo.

46. Como explicarias el deploy del frontend en una frase?**Respuesta:**

El frontend se compila con Vite usando variables del ambiente, se empaqueta en una imagen Nginx y se despliega a K3s igual que el backend, cambiando el deployment `pagina` del namespace correspondiente.

Evidencia esperada: Dockerfile multi-stage y workflows del frontend.

47. Como explicarias el deploy del backend en una frase?**Respuesta:**

El backend TypeScript se compila dentro de una imagen Node, se etiqueta con el SHA del commit, se importa a K3s y se publica actualizando el deployment `api` en `preprod` o `prod`.

Evidencia esperada: `api/Dockerfile` y workflows del backend.

48. Si el profe pregunta "donde esta Kubernetes?", que respondes?**Respuesta:**

En el repositorio no estan los manifests del cluster, pero el pipeline interactua con Kubernetes/K3s mediante `kubectl`. Eso demuestra que los deployments `api` y `pagina` ya existen en los namespaces `preprod` y `prod`. Como mejora, esos manifests deberian versionarse como infraestructura como codigo.

Evidencia esperada: comandos `kubectl set image` en workflows.

49. Como sabes que version esta corriendo?**Respuesta:**

Por la etiqueta de imagen que se puso en el Deployment. Como el pipeline usa el SHA del commit, puedo revisar la imagen del deployment y mapearla al commit exacto en GitHub.

Evidencia esperada: `kubectl describe deployment/api -n prod` mostraria `premier-api:<sha>`.

50. Que significa despliegue reproducible?**Respuesta:**

Que puedo reconstruir y desplegar la misma version de forma consistente. En este proyecto, el SHA ayuda a identificar la version, Docker define el ambiente de ejecucion y GitHub Actions define los pasos. Para que fuera mas completo, faltaria versionar tambien los manifests de Kubernetes.

Evidencia esperada: Dockerfiles, workflows y tags por SHA.

51. Como se protegen contra cambios accidentales en produccion?**Respuesta:**

La separacion de ramas ayuda: primero se puede probar en `preprod` y solo despues mezclar o subir a `main`. Idealmente se complementa con branch protection, pull requests, reviews y aprobaciones manuales para ambientes productivos.

Evidencia esperada: workflows separados por rama.

52. Que diferencia hay entre variables de build y variables de runtime?**Respuesta:**

Las variables de build se usan al construir la imagen. En el frontend, `VITE_API_URL` se queda dentro del bundle generado. Las variables de runtime se leen cuando arranca el contenedor. En backend, `SUPABASE_SERVICE_KEY` o `PORT` se leen al ejecutar Node.

Evidencia esperada: Dockerfile frontend usa `ARG` ; backend usa `process.env` .

53. Que pasa si cambia la URL de la API en frontend?**Respuesta:**

Como Vite inyecta las variables al build, hay que reconstruir y redeplegar la imagen del frontend. No basta con cambiar una variable de runtime si el valor ya quedo compilado en los archivos estaticos.

Evidencia esperada: `ARG VITE_API_URL` y `ENV VITE_API_URL=$VITE_API_URL` en Dockerfile frontend.

54. Como explicas el manejo de cache del frontend?**Respuesta:**

Nginx evita cache fuerte para rutas principales para que el usuario reciba la app actualizada, pero permite cache largo en `/assets` porque esos archivos suelen tener hash en el nombre y pueden cachearse de forma segura.

Evidencia esperada: `Cache-Control "no-cache"` en `/` y `immutable` en `/assets/` .

55. Como responder si preguntan "esta todo perfecto?"**Respuesta:**

No diria que esta perfecto; diria que el flujo base esta funcionando y cubre multientorno, imagenes, rollout y configuracion por variables. Pero tambien identifico mejoras: pruebas en pipeline, manifests versionados, observabilidad formal, escaneo de imagenes y documentacion de backups. Esa respuesta muestra criterio tecnico.

Evidencia esperada: mencionar fortalezas y gaps reales, sin inventar herramientas no existentes.

Comandos utiles para mencionar

Ver estado de pods:

```
kubectl get pods -n prod
```

```
kubectl get pods -n preprod
```

Ver rollout:

```
kubectl rollout status deployment/api -n prod
```

```
kubectl rollout status deployment/pagina -n prod
```

Rollback:

```
kubectl rollout undo deployment/api -n prod
```

```
kubectl rollout undo deployment/pagina -n prod
```

Logs:

```
kubectl logs deployment/api -n prod
```

```
kubectl logs deployment/pagina -n prod
```

Ver imagen desplegada:

```
kubectl describe deployment/api -n prod
```

```
kubectl describe deployment/pagina -n prod
```

Probar health check:

```
curl https://api.zamer-o.com/api/health
```

```
curl https://api-preprod.zamer-o.com/api/health
```

Respuestas cortas para memorizar

Si te preguntan...	Responde...
Como despliegan?	Con GitHub Actions self-hosted, Docker y K3s. El pipeline construye la imagen, la etiqueta con SHA, la importa a K3s y actualiza el Deployment.
Como separan ambientes?	Por rama, namespace y URLs. <code>preprod</code> va a namespace <code>preprod</code> ; <code>main</code> va a <code>prod</code> .
Como hacen rollback?	Con <code>kubectl rollout undo</code> o redeployando una imagen anterior identificada por SHA.
Donde estan los secretos?	El codigo los lee por variables de entorno; en Kubernetes deberian inyectarse como Secrets.
Como verifican salud?	Con <code>/api/health</code> , rollout status, estado de pods y pruebas funcionales.
Que falta mejorar?	Tests en pipeline, IaC completa, registry externo, observabilidad avanzada, escaneo de imagenes y backups documentados.

Evidencia del repo que debes saber ubicar

- `back/premier-hub-backend/.github/workflows/deploy.yml` : deploy backend a produccion.
- `back/premier-hub-backend/.github/workflows/deploy-preprod.yml` : deploy backend a preproduccion.
- `back/premier-hub-backend/api/Dockerfile` : imagen del backend.
- `back/premier-hub-backend/api/src/index.ts` : puerto, CORS, rutas y health check.
- `back/premier-hub-backend/api/src/db.ts` : conexion a Supabase con variables de entorno.
- `back/premier-hub-backend/api/src/rutas/api_auth.ts` : cookie de sesion segura.
- `front/premier-hub-frontend/.github/workflows/deploy.yml` : deploy frontend a produccion.
- `front/premier-hub-frontend/.github/workflows/deploy-preprod.yml` : deploy frontend a preproduccion.
- `front/premier-hub-frontend/pagina/Dockerfile` : build multi-stage del frontend.
- `front/premier-hub-frontend/pagina/nginx.conf` : SPA routing y cache.

Consejo para la entrevista: No inventes que tienen Terraform, Prometheus, Grafana, Helm o registry si el profe pide evidencia. Puedes decir: "Eso sería una mejora; lo que actualmente tenemos versionado es Dockerfile y GitHub Actions, y el pipeline interactúa con K3s por kubectl".